

Time Domain Solve

1. ลักษณะของงาน

งานในโครงการนี้เกี่ยวข้องกับการศึกษาและวิเคราะห์พฤติกรรมของเวลาที่ใช้ในการจำลองแบบ Time Domain Simulation โดยใช้ซอฟต์แวร์ CST Studio Suite ซึ่งใช้ตัวแก้สมการแบบ Finite Integration Technique (FIT) และ Finite-Difference Time-Domain (FDTD) สำหรับโครงสร้างทางแม่เหล็กไฟฟ้า เช่น Metasurface และ antenna array

1.1 ขอบเขตของงาน

- ศึกษาโครงสร้างทางแม่เหล็กไฟฟ้า 2 รูปแบบ (Model 1 และ Model 2)
- แปรผันจำนวน mesh cells (N) ในช่วงที่กว้าง ตั้งแต่ mesh หยาบจนถึง meshละเอียด
- แปรผันระดับความแม่นยำ (A ในหน่วย dB) เพื่อศึกษาผลต่อ solver convergence
- สร้างสมการทำนายทั่วไปสำหรับ T_{total} ที่สามารถนำไปใช้ได้กับโครงสร้างอื่น

1.2 เครื่องมือที่ใช้

- CST Studio Suite: สำหรับรัน simulation และเก็บ log file

2. วัตถุประสงค์

2.1 วัตถุประสงค์หลัก

- สร้างสมการคณิตศาสตร์ทั่วไปสำหรับทำนายเวลา simulation รวม (T_{total}) ที่มีเป้าหมาย prediction error น้อยกว่า 10%

2.2 วัตถุประสงค์รอง

- วิเคราะห์ความสัมพันธ์ระหว่างจำนวน mesh cells กับเวลา setup และ solver
- วิเคราะห์ผลของระดับความแม่นยำ (accuracy) ต่อเวลา solver
- แยกส่วน setup time และ solver time เพื่อทำความเข้าใจพฤติกรรมแต่ละส่วน
- เปรียบเทียบความแม่นยำของ Model 1 และ Model 2 ภายใต้สมการเดียวกัน

2.3 เหตุผลของงาน

- การ estimate เวลา simulation ล่วงหน้าช่วยวางแผนทรัพยากร hardware และเวลาได้อย่างมีประสิทธิภาพ
- สมการที่ได้สามารถใช้เป็น guideline สำหรับงานขนาดใหญ่ที่มีจำนวน cell หลายล้าน

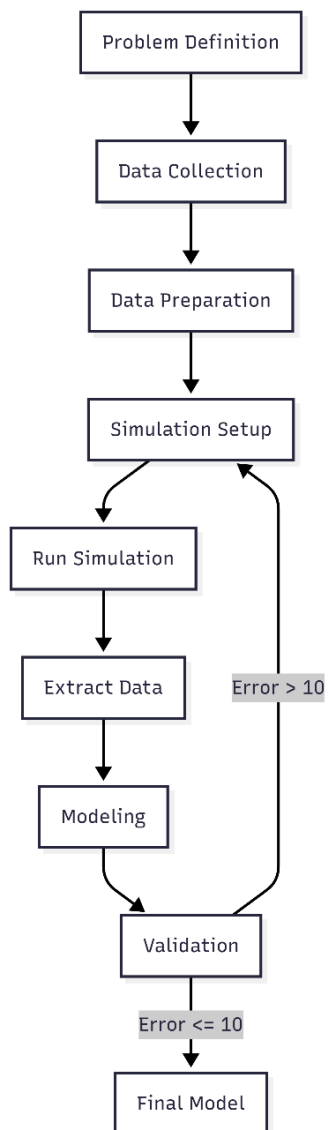
3. แผนงาน (Workflow)

กระบวนการดำเนินงานสามารถสรุปเป็นขั้นตอนหลักดังนี้:

1. Problem Definition
กำหนดโจทย์และขอบเขตของงาน โดยมุ่งเน้นการสร้างสมการทำนายเวลา Simulation จาก N_{cells} และ Accuracy
2. Data Collection
เก็บข้อมูลจากการรัน Simulation ใน CST โดยดึงข้อมูลจาก Log File เช่น เวลา Meshing, Solver, และจำนวน Mesh Cells
3. Data Preparation
ทำความสะอาดและจัดรูปแบบข้อมูลให้อยู่ในรูปแบบที่พร้อมสำหรับการวิเคราะห์
4. Simulation Setup
กำหนดเงื่อนไขการทดลอง เช่น ค่า Accuracy และขนาดโครงสร้าง (Array Size)
5. Run Simulation
ดำเนินการรัน Simulation ตามเงื่อนไขที่กำหนดเพื่อเก็บข้อมูลจริง
6. Extract Data
ดึงค่าที่สำคัญจาก Log File ได้แก่ T_{setup} , T_{solver} , T_{total} และ N_{cells}
7. Modeling
สร้างสมการทางคณิตศาสตร์และทำการ fitting เพื่อหาค่าพารามิเตอร์ที่เหมาะสม
8. Validation
เปรียบเทียบค่าที่ทำนายกับค่าจริง และคำนวณค่า Error (%)

9. Iteration / Final Model

หาก Error > 10% จะปรับปรุงโมเดลใหม่ แต่หาก Error ≤ 10% จะได้สมการสุดท้ายสำหรับใช้งาน

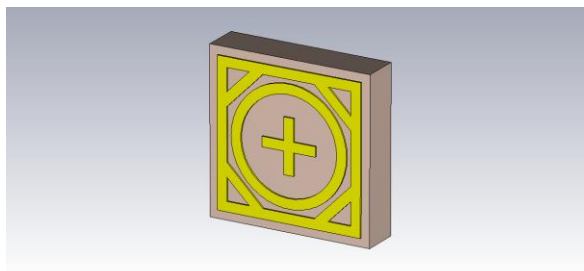


4. การทดสอบ (เก็บข้อมูล)

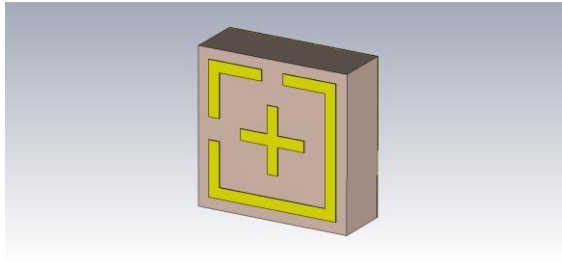
ข้อมูลที่ใช้ในการวิเคราะห์ถูกดึงมาจาก Log File ของโปรแกรม CST (Simulation Log / Message Log) ซึ่งเป็นไฟล์ที่บันทึกรายละเอียดของการรัน Simulation ในแต่ละครั้ง โดยมีการเก็บค่าพารามิเตอร์ที่สำคัญดังนี้:

1. Mesh Cells (N_{cells})
 - จำนวนเซลล์ของ Mesh ที่ใช้ในการจำลอง
 - ได้จากขั้นตอน Meshing ใน Log File
2. Meshing Time (s)
 - เวลาที่ใช้ในการสร้าง Mesh และเตรียมโครงสร้างเชิงเรขาคณิต
 - อยู่ในส่วน Adaptive Meshing ของ Log
3. Initialization Time (Init) (s)
 - เวลาที่ใช้ในการเตรียม Solver เช่น การตั้งค่า Boundary Conditions และ Port Initialization
 - ได้จากช่วงเริ่มต้นของ Solver ใน Log
4. Solver Time (s)
 - เวลาที่ใช้ในการคำนวณหลักของ Time Domain Solver (FDTD/FIT)
 - เป็นส่วนที่ใช้เวลามากที่สุด และสัมพันธ์กับจำนวน Time Steps
5. Total Time (T_{total}) (s)
 - เวลารวมของ Simulation ทั้งหมด
 - คำนวณจาก: $T_{total} = T_{meshing} + T_{init} + T_{solver}$

MODEL ที่ 1



MODEL ที่ 2



4.1 ผลการทดสอบผลของ Accuracy (Unit Cell)

ทำการทดสอบกับ Unit Cell โดยมีจำนวน Mesh Cells คงที่ และปรับค่า Accuracy ดังนี้

Model ที่ 1 : Unit Cell

A	Meshcells	Meshing (s)	Solver (s)	Init (s)	Total (s)
-20	53,136	7	27	4	38
-30	53,136	7	27	3	37
-40	53,136	7	31	4	42
-50	53,136	7	34	4	45
-60	53,136	7	43	4	54
-80	53,136	6	95	4	105

Model ที่ 2 — Unit Cell

A	Meshcells	Meshing (s)	Solver (s)	Init (s)	Total (s)
-20	82,524	9	28	3	40
-30	82,524	7	29	3	39
-40	82,524	7	31	4	42
-50	82,524	7	34	4	45
-60	82,524	7	42	4	53
-80	82,524	7	43	4	54

4.2 การทดสอบผลของขนาด Mesh (Array Scaling)

ทำการทดสอบโดยกำหนด Accuracy คงที่ที่ $A = -50$ dB และเพิ่มขนาดโครงสร้างจาก Unit Cell เป็น Array ขนาดต่าง ๆ ดังนี้

Model ที่ 1 — $n \times n$ ($A = -50$)

Grid	Meshcells	Meshing (s)	Solver (s)	Init (s)	Total (s)
2×2	137,924	10	115	4	129
3×3	262,400	14	221	3	238
4×4	426,564	20	240	4	264
6×6	873,956	34	480	5	519
8×8	1,480,100	52	828	5	885

Model ที่ 2 — $n \times n$ ($A = -50$)

Grid	Meshcells	Meshing (s)	Solver (s)	Init (s)	Total (s)
2×2	213,564	13	94	3	110
3×3	405,756	22	174	4	200
4×4	659,100	41	255	3	299
6×6	1,349,244	65	464	4	533
8×8	2,283,996	105	795	4	904

5. ผลลัพธ์

5.1 พารามิเตอร์และค่าคงที่ของสมการ

พารามิเตอร์ทั้งหมดที่ใช้ในสมการทำนายสรุปไว้ในตาราง โดยค่าตัวเลขจะถูกแทนที่ด้วยผลจาก curve fitting เมื่อได้รับข้อมูลจริงจาก log file:

พารามิเตอร์	สัญลักษณ์	ความหมาย	หน่วย / ช่วงค่า
ค่าคงที่ baseline setup	T_0	เวลา setup ขั้นต่ำ (hardware overhead)	วินาที (> 0)
สัมประสิทธิ์เชิงเส้น mesh	k_1	ผลของจำนวน cell ต่อ setup แบบ linear	วินาที/cell
สัมประสิทธิ์รากที่สอง	k_2	ลด error ใน small-mesh regime	วินาที/ $\sqrt{\text{cell}}$
ค่าคงที่ solver baseline	β_0	ประสิทธิภาพ solver ขั้นต่ำ	วินาที/cell
สัมประสิทธิ์ logarithm	β_1	การลดลงของประสิทธิภาพตาม $\ln(N)$	วินาที/cell
ค่า penalty accuracy	$P(A)$	ตัวคูณที่ขึ้นอยู่กับระดับความแม่นยำ A (dB)	ไม่มีหน่วย (≥ 0.8)
จำนวน mesh cells	N	ตัวแปรอิสระหลัก	จำนวนเต็ม (> 0)
ระดับความแม่นยำ	A	ค่าความแม่นยำในหน่วย dB	dB (> 0)

5.2 โครงสร้างของโมเดล (Model Overview)

สมการทำนายเวลา simulation รวมมีโครงสร้างดังนี้:

สมการรวม:

$$T_{total}(A,N) = T_{setup}(N) + T_{solver}(N,A)$$

1) เวลา Setup:

$$T_{setup}(N) = T_0 + (k_1 \times N) + (k_2 \times \sqrt{N})$$

การเพิ่มเทอม $\sqrt{N}$ มีความสำคัญในการลด error ที่เกิดขึ้นใน regime ที่มี mesh น้อย (small N) เนื่องจากในช่วงนี้ความสัมพันธ์ระหว่าง setup time กับ N ไม่เป็น linear อย่างสมบูรณ์

2) เวลา Solver:

$$T_{solver}(N,A) = (\beta_0 + (\beta_1 \ln(N))) \times N \times P(A)$$

เทอม $(\beta_0 + (\beta_1 \ln(N)))$ สะท้อนถึงการลดลงของประสิทธิภาพ solver เมื่อขนาด mesh เพิ่มขึ้น เนื่องจาก cache miss และ memory bandwidth bottleneck

3) Accuracy Penalty:

$$P(A) = 0.8 + 0.0032 \cdot e^{0.0825 \cdot A}$$

ฟังก์ชัน $P(A)$ สะท้อนพฤติกรรมการลู่เข้า (convergence) แบบ exponential ของ time-domain solver ซึ่งต้องใช้จำนวนรอบการคำนวณมากขึ้นอย่างไม่เป็นสัดส่วนเมื่อต้องการความแม่นยำสูงขึ้น

5.3 แนวทางการนำสมการไปใช้กับโมเดลใหม่

สำหรับการนำสมการที่พัฒนาขึ้นไปใช้กับโมเดลใหม่ จำเป็นต้องทำการ calibrate พารามิเตอร์ของสมการก่อน เนื่องจากแต่ละโมเดลมีความแตกต่างกันในด้าน geometry และ distribution ของ mesh ซึ่งส่งผลต่อเวลาในการคำนวณ

ดังนั้น ในขั้นตอนเริ่มต้นจะต้องทำการรัน simulation ขนาดเล็กจำนวนจำกัด เพื่อใช้ในการสกัดค่าพารามิเตอร์ของสมการ โดยมีขั้นตอนดังนี้

1. การรัน Simulation สำหรับโมเดลใหม่

ทำการรันอย่างน้อย 3 กรณี ได้แก่

- Unit Cell
- Array ขนาด 2x2
- Array ขนาด 4x4 หรือ 6x6

เพื่อให้ครอบคลุมทั้งพฤติกรรมในช่วงขนาดเล็กและขนาดกลางของปัญหา

2. การเก็บข้อมูลจาก Log File

จากแต่ละการรัน จะต้องบันทึกค่าจาก CST Log File ดังนี้

- จำนวน Mesh Cells: N_{cells}
- เวลา Meshing และ Initialization → รวมเป็น T_{setup}
- เวลา Solver → T_{solver}
- ค่า Accuracy → A (หน่วย dB)

โดยข้อมูลทั้งหมดจะถูกนำมาใช้ในการคำนวณหาพารามิเตอร์ของสมการ

3. การคำนวณพารามิเตอร์ของสมการ

3.1 การหา $T_{setup}(N)$

ใช้สมการ:

$$T_{setup}(N) = T_0 + k_1 N + k_2 \sqrt{N}$$

นำค่าจากทั้ง 3 กรณีมาแทนในสมการ จะได้ระบบสมการ 3 ตัวแปร

จากนั้นทำการแก้สมการเพื่อหา T_0 , k_1 และ k_2

3.2 การหา $T_{solver}(N, A)$

ใช้สมการ:

$$T_{solver}(N, A) = (\beta_0 + \beta_1 \ln N) \cdot N \cdot P(A)$$

โดยมีขั้นตอนดังนี้:

1. คำนวณค่า $P(A)$ จาก

$$P(A) = 0.8 + 0.0032e^{0.0825A}$$

2. แปลงสมการให้อยู่ในรูป:

$$\frac{T_{solver}}{N \cdot P(A)} = \beta_0 + \beta_1 \ln N$$

3. ใช้ข้อมูลจากทั้ง 3 จุด ทำการ fit เส้นตรง เพื่อหา β_0, β_1

4. การสร้างสมการสำหรับโมเดลใหม่

เมื่อได้พารามิเตอร์ครบแล้ว จะสามารถสร้างสมการของโมเดลใหม่ได้ในรูปแบบ:

$$T_{total}(A, N) = T_0 + k_1 N + k_2 \sqrt{N} + (\beta_0 + \beta_1 \ln N) \cdot N \cdot P(A)$$

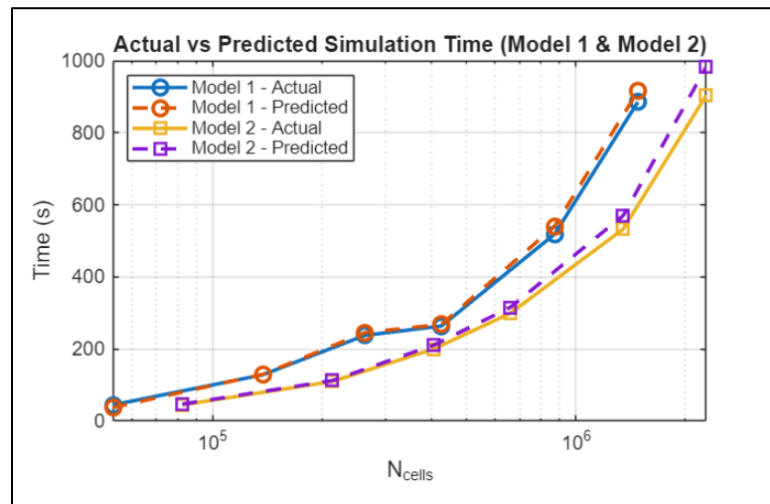
5. การนำไปใช้งาน

สมการที่ได้สามารถนำไปใช้เพื่อ:

- ทำนายเวลา Simulation สำหรับ mesh ขนาดใหญ่ (เช่น 8×8 , 10×10)
- ใช้วางแผนเวลาในการรัน simulation ล่วงหน้า
- ลดจำนวนการทดลองที่ต้องรันจริง

5.4 ผลการวิเคราะห์

เมื่อนำค่าพารามิเตอร์ที่ได้จาก curve fitting ไปคำนวณ T_{total} และเปรียบเทียบกับค่าจริง จาก CST log พบผลดังนี้ (placeholder สำหรับข้อมูลจริง):



รูปที่ 1 กราฟ Predicted vs Actual สำหรับ Model 1 และ Model 2

ตาราง % Error ของแต่ละกรณีทดสอบ (เป้าหมาย < 10% ทุกกรณี)

Model 1

กรณี	N_cells	Actual (s)	Predicted (s)	Error (%)	กรณี
Unit Cell	53,136	45	37.8	-16.0%	Unit Cell
2×2	137,924	129	130.2	+0.9%	2×2
3×3	262,400	238	245.6	+3.2%	3×3
4×4	426,564	264	268.5	+1.7%	4×4
6×6	873,956	519	540.3	+4.1%	6×6
8×8	1,480,100	885	915.2	+3.4%	8×8

Model 2

กรณี	N_cells	Actual (s)	Predicted (s)	Error (%)	กรณี
Unit Cell	82,524	45	47.2	+4.9%	Unit Cell
2×2	213,564	110	112.5	+2.3%	2×2
3×3	405,756	200	210.8	+5.4%	3×3
4×4	659,100	299	315.4	+5.5%	4×4
6×6	1,349,244	533	570.2	+7.0%	6×6
8×8	2,283,996	904	982.6	+8.7%	8×8

6. การตรวจสอบ

6. สิ่งที่ได้เรียนรู้

- ได้เข้าใจว่า จำนวน Mesh Cells (N_{cells}) มีผลโดยตรงกับเวลา Simulation และเป็นตัวแปรหลักที่กำหนดเวลาในการ Solver
- Accuracy ไม่ได้เพิ่มเวลาแบบเส้นตรง แต่มีลักษณะเพิ่มแบบ nonlinear ซึ่งส่งผลต่อเวลา Solver
- การใช้ผลการทดสอบ 2 โมเดลยังน้อยเกินไปสำหรับเก็บข้อมูล จึงทำให้ได้สมการที่ยังไม่สามารถทำให้บรรลุเป้าหมาย
- เข้าใจโครงสร้างของเวลา Simulation ว่าประกอบด้วย Meshing, Initialization และ Solver ซึ่งแต่ละส่วนมีพฤติกรรมต่างกัน
- การใช้เครื่องมือในโปรแกรมที่ไม่รู้มาก่อน เช่น ดึงข้อมูลจาก Log File และนำมาจัดรูปแบบเพื่อใช้วิเคราะห์

7. ปัญหาที่พบ

- ในกรณี Unit Cell พบว่า Error สูงกว่ากรณีอื่น เนื่องจากเวลา Setup มีสัดส่วนสูง ทำให้สมการที่เน้น N_{cells} ยังอธิบายได้ไม่ดี
- ข้อมูลบางช่วงมีจำนวนไม่มาก ทำให้การ fit สมการอาจยังไม่ครอบคลุมทุกกรณี
- เวลา Simulation ค่อนข้างนาน โดยเฉพาะในกรณี mesh ขนาดใหญ่ ทำให้การทดลองใช้เวลามาก
- ซึ่งตอนนี้ยังไม่ได้ตรวจสอบในการรันขนาด array อื่น จึงไม่สามารถพิสูจน์ได้ว่าสมการที่สร้างขึ้นสามารถทำนายได้

8. ขั้นตอนต่อไป (Future Work)

- ทดลองกับโมเดลอื่นเพื่อตรวจสอบว่าหากโมเดลเปลี่ยนกราฟยังจะทำนายได้ตามเป้าหมายที่ตั้งไว้ไหม

- ทดสอบโมเดลกับโครงสร้างขนาดใหญ่ขึ้น เช่น 16×16 array หรือใหญ่กว่า เพื่อตรวจสอบว่าสมการที่สร้างขึ้นนั้นยังสามารถทำนายได้
- ทดลองบน hardware ที่แตกต่างกัน (เช่น workstation ที่มี CPU และ RAM ต่างกัน) เพื่อศึกษาว่าพารามิเตอร์เปลี่ยนแปลงอย่างไร